DATA MODELING · DOMAIN 10 - DATA MODELING

Designing Delta tables that scale

Section 10. Where the metadata lives, how you copy a table, and one question the guide publishes.

SECTION 10 of 10 WEIGHT 6% TOPIC Data modeling

admin@sthithapragna.com

A Delta table is **files plus a log**, and a **name in a metastore**. Knowing which layer an operation touches answers a surprising number of questions.

What renaming a table actually does

- The guide's first sample question renames a table with a typo in its name, and asks what happens.
- **The published answer: the table reference in the metastore is updated.**
- Not the transaction log. Not a new log. No files are moved, dropped or recreated.
- A rename is a **metastore operation**. The data and its log are untouched.

> ON THE EXAM
>
> The distractors are all about the transaction log, and every one is wrong. Renaming is the layer above.

The partition column question

- A posts table with user id, post text, post id, longitude, latitude, **post time as a timestamp**, and **date as a date**.
- The question: which column should be used for partitioning? **The published answer is date.**
- Not **post id**, which is unique, so every row becomes its own partition.
- Not **user id**, which is high cardinality. Not **post time**, a timestamp granular to the second.

> ON THE EXAM
>
> This is the objective's own published example. Partition-column reasoning is still tested, even though liquid clustering now supersedes partitioning.

What makes a column a good partition key

PROPERTY	WHY IT MATTERS
Low cardinality	Few distinct values means few, large partitions. Unique columns are the worst possible choice.
Queries filter on it	A partition only helps if queries actually restrict on that column.
Roughly even distribution	Wildly uneven partitions reproduce the skew problem from module 13.
Partitions of useful size	Databricks guidance: at least about 1 GB per partition.

ON THE EXAM

A date column usually satisfies all four. A timestamp satisfies the second and fails the first three.

But partitioning is not the modern answer

- Module 22 established that **liquid clustering replaces partitioning and ZORDER**, and is recommended for all new tables.
- Yet the guide's own sample question asks you to choose a partition column, and publishes an answer.
- So learn **both**. Answer the partitioning question if asked, and know clustering is the current recommendation.
- Note the sample questions are described as **retired from a previous version** of the exam.

> ON THE EXAM
>
> An honest tension rather than a contradiction. The reasoning behind a good partition key is still the reasoning behind a good clustering key.

Deep and shallow clones

- **Deep clone copies the data**, producing a complete independent table. **It is the default.**
- **Shallow clone copies the definition and references the source's files** without copying them.
- So a shallow clone is nearly instant and nearly free, whatever the size of the source.
- And it is **not independent** of the source's files, which matters for retention and lifecycle.

> ON THE EXAM
>
> Deep being the default is worth remembering. If a question shows CLONE with neither keyword, it is a deep clone.

What can be cloned, and how

SOURCE	SUPPORT
Delta	Deep and shallow.
Parquet	Deep and shallow.
Foreign Iceberg	Deep and shallow.
Managed Iceberg	Deep only, and the format cannot change during cloning.
Streaming tables and materialized views	Not supported as source or target.

ON THE EXAM

Shallow clone on Unity Catalog managed tables needs DBR 13.3 LTS or above. Below that there is no support for it in Unity Catalog at all.

When each clone is right

- **Shallow** for short-lived experiments, testing a migration, or trying a change against production-shaped data.
- **Deep** for anything that must outlive the source, or survive independently of it.
- A shallow clone whose source is vacuumed aggressively is a shallow clone with a lifespan.
- Deep clone is also how you take a genuine backup or move a table between environments.

> ON THE EXAM
>
> The question to ask is how long the copy must live, and whether it must survive the source being cleaned up.

Cloning a table as it used to be

- The clone source may include a **temporal specification**, so you can clone a table **as at a past version**.
- That gives you a named, queryable table holding a historical state, rather than a query with a version clause.
- Useful for reproducing an old report, or investigating what changed between two versions.
- Bounded by retention. You can only clone as far back as the files still exist.

> ON THE EXAM
>
> Clone plus time travel is more useful than either alone. A version becomes an object you can grant on and query normally.

How far back you can actually go

- Query a past version by **version number** or by **timestamp**.
- Bounded by two things: **history retention**, defaulting to 30 days, and **vacuum**, defaulting to 7.
- Once vacuum has removed the files, the version cannot be read however recent the log entry is.
- For longer reproducibility, set the retention property **before** enabling automatic maintenance.

> ON THE EXAM
>
> Module 26's two numbers again. History retention governs the log; vacuum retention governs the files, and files win.

MERGE, and what it does not know

- **MERGE** performs an upsert: update where a key matches, insert where it does not, optionally delete.
- It is the standard way to apply changes to a dimension, and it is **atomic**.
- But it applies changes **in the order they arrive**, which is module 8's whole argument.
- For an out-of-order change feed, **AUTO CDC** exists precisely because MERGE cannot sequence.

> ON THE EXAM
>
> MERGE is right when you control the ordering. It is wrong when a source can deliver a correction before the thing it corrects.

Constraints that the table enforces

- **NOT NULL** and **CHECK** constraints are enforced by the table itself. A violating write fails.
- That is stronger than an expectation, which lives in a pipeline and only governs data arriving through it.
- And blunter. A failed write is a failed job, with no warn or drop option.
- Expectations from module 14 offer three responses. A constraint offers one.

> ON THE EXAM
>
> Two mechanisms with different reach. A constraint protects the table from every writer; an expectation protects one pipeline's output.

Designing for a schema that will move

- Delta supports schema evolution, so a table's shape can change without rebuilding it.
- Module 21's caution applies: **a renamed column is treated as a new column added**, with the old one nulled for new rows.
- A dropped column is a **soft delete**, null for new rows rather than removed.
- So design for addition, which is cheap, rather than for renaming, which is not.

> ON THE EXAM
>
> Scalable table design is largely about which changes are cheap. Adding columns is; renaming and reshaping are not.

What to carry into the exam

- **Renaming a table updates the metastore**, not the transaction log. Published answer in the guide.
- **Partition on low cardinality columns that queries filter on.** The guide's answer is the date column, not the timestamp.
- **DEEP CLONE is the default.** Shallow references the source's files. Streaming tables and MVs cannot be cloned.
- Clone accepts a version, so you can clone a table as it used to be.
- **MERGE applies arrival order.** Constraints are enforced by the table; expectations by the pipeline.

> ON THE EXAM
>
> Two of these come from the guide's own published sample questions, which makes them the highest-confidence facts in the course.

> A table of user posts has columns for user id, post text, post id, longitude, latitude, a post time timestamp, and a date column.
>
> The team must choose a partitioning column. The table receives several million posts per day and analysts almost always filter by day.

Which column should be used?

A post_id

B post_time

C date

D user_id

◆ PAUSE AND DECIDE

Which column should be used?

A post_id

Unique per row, so partitioning on it creates one partition per post. The worst possible choice.

B post_time

A timestamp granular to the second is effectively unique at several million posts per day.

C date

Low cardinality, evenly distributed, and exactly what analysts filter on. The guide's published answer.

D user_id

High cardinality, and unevenly distributed since some users post far more than others.

ON THE EXAM

This is the guide's own sample question and published answer. Both wrong date-like options fail on cardinality, which is the first test.

> A table was created with a typo in its name. An engineer runs an ALTER TABLE statement to rename it, then worries that the rename may have rewritten the underlying data or invalidated existing time travel history.
>
> The table holds 40 TB.

What actually happened?

A All related files and metadata were dropped and recreated in a single transaction.

B The name change was recorded in the Delta transaction log.

C A new transaction log was created for the renamed table.

D The table reference in the metastore was updated.

◆ PAUSE AND DECIDE

What actually happened?

A All related files and metadata were dropped and recreated in a single transaction.

Nothing is rewritten. A 40 TB rename returns immediately, which is the practical tell.

B The name change was recorded in the Delta transaction log.

The log records changes to the table's data and metadata, not the name it is known by.

C A new transaction log was created for the renamed table.

The existing log is untouched, which is why time travel history survives.

D **The table reference in the metastore was updated.**

A rename is a metastore operation. The guide's published answer.

ON THE EXAM

The engineer's worry answers itself: a 40 TB rename that returns instantly cannot have moved any data.

> A team needs a copy of a 12 TB production table to test a destructive schema migration. The copy is needed for two days and will then be discarded. Production must not be affected.
>
> They plan to use a create-table-as-select.

What is the better approach?

A. A shallow clone, which copies the definition and references the source files, and is nearly instant.

B. A deep clone, since the migration is destructive and must not touch production files.

C. The create-table-as-select is correct; cloning would link the copy to production.

D. Time travel, querying production at a past version instead of copying.

◆ PAUSE AND DECIDE

What is the better approach?

A A shallow clone, which copies the definition and references the source files, and is nearly instant.

Two days, then discarded. Writes to the clone create new files and never modify the source's.

B A deep clone, since the migration is destructive and must not touch production files.

Also safe, and copies 12 TB for a copy that lives two days.

C The create-table-as-select is correct; cloning would link the copy to production.

Copies 12 TB, takes hours, and loses constraints and properties that a clone preserves.

D Time travel, querying production at a past version instead of copying.

Time travel is read-only. You cannot test a destructive migration against it.

ON THE EXAM

B is safe and expensive. The deciding facts are the two-day lifespan and that writing to a shallow clone never alters the source's files.

> An engineer wants to snapshot a materialized view before a pipeline change, so they can compare results afterwards. They attempt a deep clone of the materialized view and it fails.
>
> They ask whether the syntax is wrong.

What should they be told?

A Deep clone requires the LOCATION clause when the source is a materialized view.

B Streaming tables and materialized views are not supported as a source or target for CLONE.

C Materialized views can only be shallow cloned.

D The clone must specify a version, since materialized views have no current version.

◆ PAUSE AND DECIDE

What should they be told?

A Deep clone requires the LOCATION clause when the source is a materialized view.

LOCATION is optional and creates an external table. It is not the blocker.

B Streaming tables and materialized views are not supported as a source or target for CLONE.

A stated limitation. The syntax is fine; the source type is not supported.

C Materialized views can only be shallow cloned.

Neither kind is supported for them.

D The clone must specify a version, since materialized views have no current version.

Invented. Materialized views do have versions.

ON THE EXAM

The realistic alternative is a create-table-as-select from the view, which produces an ordinary table they can compare against later.

> A dimension table is maintained with a MERGE statement matching on customer id, run against a change feed from an operational system. The feed can deliver events out of order after network interruptions.
>
> Auditors find dimension rows holding older values than ones recorded earlier.

What is the cause and the remedy?

A. MERGE applies changes in arrival order. Use AUTO CDC, which sequences by a column you nominate.

B. The MERGE is matching on the wrong key; add the event timestamp to the match condition.

C. MERGE is not atomic, so concurrent writes are interleaving.

D. Enable change data feed on the dimension table so ordering is preserved.

◆ PAUSE AND DECIDE

What is the cause and the remedy?

A MERGE applies changes in arrival order. Use AUTO CDC, which sequences by a column you nominate.

Module 8's argument exactly: MERGE has no concept of when a change happened, only when it arrived.

B The MERGE is matching on the wrong key; add the event timestamp to the match condition.

Would make every event a distinct row rather than a version of one.

C MERGE is not atomic, so concurrent writes are interleaving.

MERGE is atomic. Ordering is the problem, not atomicity.

D Enable change data feed on the dimension table so ordering is preserved.

CDF records changes to a table. It does not order incoming events.

ON THE EXAM

An older value overwriting a newer one is the signature. The fix is sequencing, which MERGE has no way to express.

> A finance team must be able to reproduce a report exactly as it was produced on any month end for the past year. They plan to rely on time travel against the source tables.
>
> The tables are Unity Catalog managed with predictive optimization enabled and default retention.

What is the problem, and what would work better?

A Nothing; time travel retains a year of history by default.

B Vacuum removes the files after the default window, so year-old versions are unreadable. Deep clone each month end into a named table.

C Shallow clone each month end, which is cheaper and equally durable.

D Enable change data feed, which preserves every version indefinitely.

◆ PAUSE AND DECIDE

What is the problem, and what would work better?

A Nothing; time travel retains a year of history by default.

History defaults to 30 days and vacuum removes files after 7. Neither is a year.

B Vacuum removes the files after the default window, so year-old versions are unreadable. Deep clone each month end into a named table.

A deep clone is independent of the source's files, so it survives vacuum. Twelve durable snapshots.

C Shallow clone each month end, which is cheaper and equally durable.

A shallow clone references the source's files, which are exactly what vacuum removes.

D Enable change data feed, which preserves every version indefinitely.

CDF is explicitly not a permanent record, and vacuum deletes its files too.

ON THE EXAM

C is the trap. Cheaper, and it depends on precisely the files the source is entitled to delete.